

CTR Analytics Platform Scale-Up Roadmap

Detailed implementation plan to evolve the current 1M-row PostgreSQL warehouse into an Airflow-orchestrated, PySpark-enabled, production-style batch analytics platform.

Project context: Criteo display advertising dataset | **Current state:** PostgreSQL warehouse on 1M rows | **Target state:** automated 5M+ pipeline with orchestration, monitoring, and ML-ready outputs

1. Executive Summary

The current project already has a strong layered PostgreSQL foundation. To make it feel genuinely industry-grade, the next evolution should be to convert it from a manually executed warehouse build into a batch analytics platform with orchestration, metadata-driven processing, scalable transformation, operational monitoring, and benchmarked production behavior.

- Keep PostgreSQL as the serving warehouse, marts layer, quality store, and reporting source for Power BI.
- Introduce PySpark for heavy batch parsing, standardization, feature preparation, and pre-aggregation at larger scale.
- Use Apache Airflow as the orchestration layer for scheduling, dependency management, retry logic, and run visibility.
- Add pipeline metadata, batch identifiers, benchmark logging, and operational dashboards so the project tells a real platform story rather than only a SQL story.
- Scale the development baseline from 1M to 5M rows first, then leave the architecture ready for optional full-file execution.

Recommended strategy: do not replace the current warehouse. Layer Airflow, PySpark, and operational metadata on top of the current build so the project grows in maturity without losing the SQL depth you already built.

2. Why Airflow Is the Right Upgrade

Airflow is a strong fit because this project is fundamentally a batch pipeline with clear stage dependencies, measurable outputs, and rerunnable data products. It adds operational discipline without changing the core warehouse logic.

- Task orchestration: define raw load, staging build, warehouse build, marts refresh, advanced SQL refresh, and quality checks as explicit DAG tasks.
- Scheduling: run on demand, on file arrival, or on a daily schedule once the project is fully automated.
- Retries and alerting: automatically retry transient failures and surface failed stages cleanly.
- Run history: preserve execution logs, durations, status, and dependency lineage.

- Parameterization: rerun the same DAG for 100K, 1M, 5M, or full data by changing one batch parameter.

Airflow should orchestrate the pipeline; it should not contain the business logic itself. The transformation logic should continue living in SQL files, PySpark jobs, and Python utility scripts so each layer stays testable and reusable.

3. Target Architecture

Layer	Primary Tool	Role In Final Platform
Raw landing	Filesystem + batch registry	Stores incoming source files and registers each batch for processing.
Heavy transformation	PySpark	Parses large files, standardizes missing values, creates derived fields, and prepares batch outputs.
Serving warehouse	PostgreSQL	Stores raw, staging, warehouse, marts, quality, and feature-store tables.
Orchestration	Airflow	Coordinates all pipeline tasks, dependencies, retries, logs, and schedules.
Monitoring	Quality schema + ops schema	Captures validation runs, pipeline runs, benchmark timings, and batch status.
Reporting	Power BI	Consumes stable mart outputs and quality summaries.
ML preparation	Feature store tables	Publishes CTR training features for future modeling.

- A new batch should flow through a registry table before any downstream task starts.
- Every major table should become batch-aware by carrying a batch identifier and source file context.
- Every orchestrated run should write to ops monitoring tables in addition to the existing quality and load audit tables.

4. Implementation Principles

- 1 Evolve, do not rewrite. Keep the current SQL warehouse and extend it with orchestration and scalable processing.
- 2 Make every stage batch-aware. If a row exists in raw, staging, warehouse, marts, or quality, it should be traceable to a batch id.

- 3 Prefer parameter-driven execution over duplicated scripts so 1M and 5M runs use the same code path.
- 4 Keep PostgreSQL focused on serving, joins, marts, and monitoring while PySpark handles the most expensive file and row processing work.
- 5 Record every run, duration, row count, and outcome so scale-up claims are backed by evidence.

5. Detailed Step-by-Step Roadmap

Phase 0: Environment and baseline hardening

- Confirm Dockerized PostgreSQL settings are stable for larger loads, especially storage availability and WAL behavior.
- Create a dedicated Python environment named `adv` if one is needed for Airflow and PySpark dependencies.
- Add a central configuration pattern for sample path, batch id, row target, database credentials, and output locations.
- Benchmark the current 1M pipeline end to end so you have a before-and-after baseline once scale changes are introduced.
- Freeze the current working SQL outputs as the reference behavior that future automated runs must continue to match.

Phase 1: Introduce operational metadata

- Create a new schema such as `ops` for orchestration metadata and run control.
- Add `ops.batch_registry` with columns like `batch_id`, `source_file`, `expected_row_count`, `actual_row_count`, `batch_status`, `created_at`, `started_at`, `completed_at`, and `notes`.
- Add `ops.pipeline_runs` to capture each Airflow-triggered pipeline execution with `dag_run_id`, `batch_id`, `target_scale`, `run_status`, `started_at`, `completed_at`, and `duration_seconds`.
- Add `ops.pipeline_steps` to capture task-level execution metrics including `task_name`, `layer_name`, `rows_processed`, `runtime_seconds`, `status`, and `error_message`.
- Update existing raw, staging, warehouse, marts, and quality objects so they can carry `batch_id` cleanly where appropriate.

This phase is one of the most important upgrades. Without batch metadata and run logging, the project looks like a manual build. With them, it starts looking like a platform.

Phase 2: Refactor scripts into parameterized pipeline components

- Refactor the current scripts so each accepts a batch id, source file, and scale target as parameters instead of assuming one fixed working file.
- Keep natural names and readable entry points, but standardize interfaces across scripts.
- Separate orchestration from logic: Airflow should call these scripts rather than embedding all transformation logic directly in DAG code.
- Standardize logging output so each script prints start time, end time, rows processed, and final status in a consistent format.
- Make every script idempotent or at least safe to rerun for the same batch after a controlled cleanup.

Phase 3: Add PySpark batch processing

- Use PySpark to read train.txt or criteo_5m.csv efficiently and apply scalable parsing for numeric and categorical fields.
- Move the most expensive row-level work into Spark: type casting, missing-value standardization, derived field creation, event batch tagging, and feature-ready exports.
- Write PySpark outputs either to intermediate processed files or directly into PostgreSQL-ready staging files depending on local performance.
- Keep the transformation rules aligned with the existing staging logic so Spark and SQL outputs remain consistent.
- Document the exact split of responsibilities: Spark for batch compute, PostgreSQL for warehouse serving and marts.

Phase 4: Build the Airflow DAG

- Stand up Airflow with Docker Compose so the environment is reproducible locally.
- Create one main DAG, for example ctr_batch_pipeline, with batch-driven parameters.
- Use task groups for landing, sampling, ingestion, staging, warehouse, marts, advanced SQL, quality, and benchmark reporting.
- Use sensors or a batch registry poll step so the DAG begins only when a target raw batch is available.
- Configure retries, failure callbacks, and clear task dependencies so the run graph looks production-ready in the Airflow UI.

Suggested Airflow task order

- 1 register_batch
- 2 validate_source_file
- 3 run_sampling_if_needed
- 4 run_pyspark_preprocessing

- 5 load_raw_layer
- 6 build_staging_layer
- 7 build_warehouse_layer
- 8 build_marts_layer
- 9 build_advanced_sql_assets
- 10 run_quality_framework
- 11 capture_benchmarks
- 12 mark_batch_complete

Phase 5: Scale the platform to 5M rows

- Register the 5M sample as a new batch in the batch registry rather than treating it as a one-off manual file.
- Run the full Airflow pipeline for the 5M batch and capture timings at each stage.
- Measure raw load duration, staging build duration, warehouse build duration, mart refresh time, advanced SQL refresh time, and quality run time.
- Track database size growth for raw, staging, warehouse, marts, and indexes.
- Tune only the stages that show real bottlenecks rather than guessing in advance.

Phase 6: Performance tuning and production hardening

- Add or revise indexes based on actual heavy joins and refresh queries observed at 5M scale.
- Consider partitioning by batch_id or derived event_day_number where it improves maintenance or load isolation.
- Ensure materialized view refreshes are used where they provide real speedups for repeated analytic queries.
- Avoid fully populating optional bridge tables unless a downstream use case justifies the extra storage and runtime.
- Review PostgreSQL memory, temp file, and WAL settings if local hardware allows tuning.

Phase 7: Expand monitoring and quality

- Keep the current quality framework, but tie each validation result explicitly to pipeline runs and batch ids.
- Add pipeline-health summaries such as total tasks, successful tasks, failed tasks, warnings, and runtime drift over time.
- Create quality trend marts or views so Power BI can show validation history instead of only latest status.

- Add data-drift style checks for CTR changes, missing-rate shifts, and row-count anomalies across batches.
- Store benchmark metrics in persistent monitoring tables rather than only console logs.

Phase 8: Formalize the feature store

- Create a dedicated feature_store schema or keep a clearly named feature-store table in marts if you prefer one serving area.
- Version the feature output by batch and feature-generation timestamp.
- Document each feature clearly, including definition, source layer, and intended ML usage.
- Keep the feature store reproducible from warehouse and mart inputs so it fits the same batch lineage model.
- Prepare the final table for Logistic Regression, XGBoost, and LightGBM style experiments even if model training remains a future phase.

Phase 9: Dashboard and portfolio finish

- Add Power BI pages for executive KPIs, feature analytics, data quality monitoring, and pipeline performance monitoring.
- Use the new quality dashboard summary and pipeline benchmark tables as direct dashboard sources.
- Update the README so it explains the platform architecture, orchestration story, and scale-up evidence clearly.
- Add a final benchmark report that compares 1M and 5M execution behavior.
- Write resume bullets that emphasize orchestration, scalable transformation, batch observability, and ML-ready outputs.

6. Recommended Build Order

- 1 Add ops schema and metadata tables.
- 2 Refactor current scripts for batch-aware parameterization.
- 3 Add PySpark preprocessing for the 5M path.
- 4 Create Airflow services in Docker Compose.
- 5 Build the first DAG for the current 1M pipeline.
- 6 Run the DAG successfully on 1M and confirm outputs match the existing manual build.
- 7 Promote the same DAG to a registered 5M batch run.
- 8 Capture benchmarks and tune the slowest tasks.
- 9 Extend monitoring, feature store, and dashboard outputs.

10 Document the final platform as a production-style analytics system.

7. Suggested Repository Additions

Area	Suggested additions
Docker	Airflow services, scheduler, webserver, worker or local executor configuration, and shared volumes for dags and logs.
DAGs	A dags directory containing ctr_batch_pipeline.py and reusable task helpers.
PySpark	A spark_jobs directory for preprocessing, batch standardization, and feature preparation jobs.
SQL	Additional ops and monitoring SQL files such as batch registry, pipeline metrics, and benchmark views.
Scripts	Batch-aware wrappers that Airflow can call reliably with parameters.
Docs	Architecture diagrams, Airflow runbook, benchmark report, and feature definitions.

8. Key Risks and How To Handle Them

- Airflow can add setup complexity quickly. Keep the first DAG narrow and focused on the pipeline you already trust.
- PySpark can become tool inflation if it is used everywhere. Use it for large-scale batch processing, not for simple warehouse queries that PostgreSQL already handles well.
- Disk pressure will continue to matter at 5M. Benchmark storage growth during each run and keep raw, processed, and database storage under control.
- The project can become too broad if cloud services, streaming, and too many extra tools are added at once. Depth is better than tool sprawl.
- Optional bridge-table population is still a cost center. Leave it optional unless a concrete analytical use case requires it.

9. Definition of Done For The Scaled Platform

- A registered 5M batch can be processed end to end from ingestion through marts and quality checks using an Airflow DAG.
- PySpark handles the heavy preprocessing path and hands clean outputs to the PostgreSQL pipeline.

- Every batch and every pipeline run is traceable through metadata tables, quality tables, and benchmark logs.
- Power BI can read stable analytical and operational outputs from PostgreSQL.
- The feature store is refreshed from the same orchestrated pipeline and tied to batch lineage.
- The README and project documentation clearly explain the architecture, tradeoffs, benchmark results, and final production-style capabilities.

10. Immediate Next Actions

- 1 Create the ops schema and pipeline metadata tables first.
- 2 Refactor the existing Python entry points so they accept batch-aware parameters.
- 3 Add Airflow to docker-compose and stand up the local Airflow environment.
- 4 Design the first DAG around the existing 1M workflow before touching 5M.
- 5 Add the PySpark preprocessing job once orchestration and metadata are in place.
- 6 Promote the pipeline to 5M, benchmark it, and use those results to guide tuning.